

MOVIMIENTO EN 3D — Personajes y NPCs

1. Dash hacia enemigo con offset en 3D

Player (GameObject con Transform en una escena 3D)

```
Vector3 direction = (enemy.position - player.position).normalized;
// separarte 1 unidad del enemigo
Vector3 offset = direction * 1.0f;
Vector3 targetPos = enemy.position - offset;

player.DOMove(targetPos, 0.2f).SetEase(Ease.InSine).OnComplete(() =>
{
    // volver a la posición inicial
    player.DOMove(startingPos, 0.3f).SetEase(Ease.OutQuad);
});
```

Detalles:

- ★ `direction` es el vector de dirección normalizado entre el jugador y el enemigo.
 - ★ Se resta un `offset` al destino para que el personaje no "se monte" sobre el enemigo.
 - ★ Perfecto para ataques cuerpo a cuerpo.
-

2. Sacudida de enemigo al ser golpeado (en 3D)

Enemy (GameObject con Rigidbody y Collider)

```
// Efecto en mundo 3D
enemy.DOShakePosition(0.2f, 0.3f);
enemy.DOScale(Vector3.one * 1.2f, 0.1f).SetLoops(2, LoopType.Yoyo);
```

Detalles:

- El `DOShakePosition` simula un temblor en el mundo 3D.
- El `DOScale` da un "rebote" corporal leve por el impacto.
- Puedes aplicarlo sobre un "mesh holder" si no quieres mover colisionadores.

CÁMARA Y CINEMACHINE

Sacudida de cámara con Cinemachine

CinemachineVirtualCamera con componente

CinemachineBasicMultiChannelPerlin:

```
CinemachineBasicMultiChannelPerlin perlin =  
virtualCam.GetCinemachineComponent<CinemachineBasicMultiChannelPerlin>();  
perlin.m_AmplitudeGain = 1.5f;  
  
DOTween.DelayedCall(0.3f, () => {  
    perlin.m_AmplitudeGain = 0f;  
});
```

Detalles:

- ★ **m_AmplitudeGain** controla la intensidad de la sacudida.
- ★ Ideal para golpes críticos o explosiones.
- ★ Puedes encapsularlo en una función para reutilizarlo fácilmente.

Zoom dramático con Cinemachine

CinemachineVirtualCamera

```
float originalFOV = virtualCam.m_Lens.FieldOfView;  
  
DOTween.To(() => virtualCam.m_Lens.FieldOfView, x =>  
virtualCam.m_Lens.FieldOfView = x, 30f, 0.5f)  
    .SetEase(Ease.OutSine)  
    .OnComplete(() => {  
        DOTween.To(() => virtualCam.m_Lens.FieldOfView, x =>  
virtualCam.m_Lens.FieldOfView = x, originalFOV, 0.5f);  
    });
```

Detalles:

- ★ Cambia el FOV de la cámara para acercar la acción.
- ★ Compatible con animaciones o efectos especiales.

EFECTOS VISUALES Y HABILIDADES

Proyectil que viaja en curva aleatoria (Path 3D)

Projectile (Prefab o instancia con Transform)

```
Vector3 start = caster.position;
Vector3 end = target.position;

// Punto medio elevado en el eje Y con variación aleatoria en X y Z
Vector3 mid = (start + end) / 2f + new Vector3(
    Random.Range(-1f, 1f),
    Random.Range(2f, 4f),
    Random.Range(-1f, 1f)
);

// Crear curva
Vector3[] path = new Vector3[] { start, mid, end };

projectile.DOPath(path, 1.0f, PathType.CatmullRom)
    .SetLookAt(0.01f)
    .SetEase(Ease.InOutSine);
```

Detalles:

- ★ **mid** es un punto medio elevado y desplazado para crear una trayectoria arqueada única.
- ★ **DOPath** usa CatmullRom para suavidad.
- ★ **SetLookAt** hace que el proyectil mire en la dirección de su movimiento.
- ★ Ideal para magia o flechas con estilo.

Teleportación mágica (desaparece → se mueve → reaparece)

Player (GameObject con malla y materiales)

```
Transform player = this.transform;

player.DOScale(Vector3.zero, 0.3f).OnComplete(() => {
    player.position = teleportTarget.position;
    player.DOScale(Vector3.one, 0.3f);});
```

Alternativas:

- ★ Usar Shader que permita desvanecimiento (**DOFade**) si tienes `Material.SetFloat("_Alpha", val)`.
- ★ Puedes reproducir un efecto de partículas entre desaparición y reaparición.

ENTORNO Y ESCENA

Puerta o compuerta que se abre con rebote (rotación)

Door (GameObject con bisuegra visible o eje local correcto)

```
door.DOLocalRotate(new Vector3(0f, 90f, 0f), 1f,
RotateMode.LocalAxisAdd)
    .SetEase(Ease.OutBounce);
```

Detalles:

- ★ Usa **DOLocalRotate** si el pivote está bien ajustado.
- ★ Puedes hacerlo acumulativo con **RotateMode.LocalAxisAdd**.

NPC que te ve y gira solo en eje Y (3D)

NPC (GameObject con cuerpo orientado en Y, como un personaje clásico)

```
Vector3 lookAt = new Vector3(player.position.x, npc.position.y,
player.position.z);
npc.DOLookAt(lookAt, 0.5f, AxisConstraint.Y);
```

Detalles:

- ★ Mantiene al NPC girando solo en eje Y (horizontal).
- ★ Ideal para reactividad simple cuando entras en un rango.

¿CÓMO CALCULAMOS UN PUNTO INTERMEDIO ALEATORIO EN 3D?

Método general:

```
Vector3 mid = (start + end) / 2f;
mid += new Vector3(
```

```

    Random.Range(-1f, 1f), // aleatorio en X
    Random.Range(2f, 4f),  // elevado en Y
    Random.Range(-1f, 1f)  // aleatorio en Z
);

```

Alternativas:

- ★ Para trayectorias siempre elevadas: solo eleva en Y, sin variar X/Z.
- ★ Para proyectiles mágicos: añade más de un punto intermedio para trayectorias serpenteantes.

¿Perros, podría ser interesante un EffectManager static?

Sí, rotundamente. Podemos tener un script que se dedique a desplegar efectos o tween. Lo veremos más abajo pero aquí teneis una pincelada:

```

EffectManager3D.PlayDashToTarget(player, enemy);
EffectManager3D.ShakeCamera(virtualCam);
EffectManager3D.CurveProjectile(projectile, start, target);

```

```

=====

using UnityEngine;
using DG.Tweening;
using Cinemachine;

public static class EffectManager3D
{
    // Dash del jugador hacia un objetivo, con un offset para no montarse sobre el enemigo
    public static void PlayDashToTarget(Transform player, Transform target, float offset = 1.0f)
    {
        Vector3 direction = (target.position - player.position).normalized; // Dirección normalizada entre
jugador y objetivo
        Vector3 dashTarget = target.position - direction * offset; // Posición destino con offset hacia atrás

        // Movimiento hacia el objetivo
        player.DOMove(dashTarget, 0.2f).SetEase(Ease.InSine).OnComplete(() =>
        {
            // Vuelve a la posición inicial
            player.DOMove(player.position - direction * offset, 0.3f).SetEase(Ease.OutQuad);
        });
    }

    // Sacudida de la cámara utilizando Cinemachine y Perlin Noise
    public static void ShakeCamera(CinemachineVirtualCamera cam, float intensity = 1.5f, float
duration = 0.3f)
    {

```

```

    var perlin = cam.GetCinemachineComponent<CinemachineBasicMultiChannelPerlin>();
// Obtener componente de ruido
    perlin.m_AmplitudeGain = intensity; // Activar sacudida

    // Detener sacudida tras cierta duración
    DOVirtual.DelayedCall(duration, () =>
    {
        perlin.m_AmplitudeGain = 0f;
    });
}

// Lanzamiento de proyectil en curva con punto intermedio aleatorio elevado
public static void CurveProjectile(Transform projectile, Vector3 start, Vector3 end, float duration =
1.0f)
{
    Vector3 mid = (start + end) / 2f + new Vector3(
        Random.Range(-1f, 1f), // Variación aleatoria en X
        Random.Range(2f, 4f), // Elevación en Y
        Random.Range(-1f, 1f) // Variación aleatoria en Z
    );

    Vector3[] path = new Vector3[] { start, mid, end }; // Definir el camino curvo con 3 puntos

    // Movimiento a lo largo del path
    projectile.DOPath(path, duration, PathType.CatmullRom)
        .SetLookAt(0.01f) // Hacer que mire hacia delante
        .SetEase(Ease.InOutSine);
}

// Desaparecer y teletransportarse (efecto de teleportación)
public static void TeleportEffect(Transform obj, Vector3 destination)
{
    // Escalar a cero para simular desaparición
    obj.DOScale(Vector3.zero, 0.3f).OnComplete(() =>
    {
        obj.position = destination; // Mover instantáneamente
        obj.DOScale(Vector3.one, 0.3f); // Reaparecer con escala normal
    });
}

// Zoom dramático con Cinemachine
public static void ZoomCamera(CinemachineVirtualCamera cam, float targetFOV = 30f, float
duration = 0.5f)
{
    float originalFOV = cam.m_Lens.FieldOfView; // Guardar FOV original

    // Reducir FOV para simular zoom
    DOTween.To(() => cam.m_Lens.FieldOfView, x => cam.m_Lens.FieldOfView = x, targetFOV,
duration)
        .SetEase(Ease.OutSine)
        .OnComplete(() =>
        {

```

```

        // Volver al FOV original
        DOTween.To(() => cam.m_Lens.FieldOfView, x => cam.m_Lens.FieldOfView = x,
originalFOV, duration);
    });
}

// Efecto de golpe para un enemigo (sacudida + rebote de escala)
public static void HitReaction(Transform enemy)
{
    // Sacudida de posición en 3D
    enemy.DOShakePosition(0.2f, 0.3f);

    // Rebote visual de escala
    enemy.DOScale(Vector3.one * 1.2f, 0.1f).SetLoops(2, LoopType.Yoyo);
}

// NPC gira hacia el jugador solo en eje Y
public static void LookAtPlayerY(Transform npc, Transform player)
{
    Vector3 lookAt = new Vector3(player.position.x, npc.position.y, player.position.z); // Mantener Y
del NPC
    npc.DOLookAt(lookAt, 0.5f, AxisConstraint.Y); // Girar solo en Y
}

// Rotación de puerta (o cualquier objeto con bisagra) con efecto de rebote
public static void RotateDoor(Transform door, float angle = 90f, float duration = 1f)
{
    // Rotación local acumulativa en eje Y
    door.DOLocalRotate(new Vector3(0f, angle, 0f), duration, RotateMode.LocalAxisAdd)
        .SetEase(Ease.OutBounce);
}
}

```

Ya tenemos un manager de efectos que puede ser llamado de la siguiente forma:

```

EffectManager3D.PlayDashToTarget(playerTransform, enemyTransform);
EffectManager3D.CurveProjectile(proyectil, origen, destino);

```

=====

¿Y podríamos tener uno para partículas y demás??

También. Os adjunto el código de un administrador de efectos de partículas y otros...ojo, también es static por lo tanto NO necesitamos “crear una variable” para utilizarlo.

```

using UnityEngine;
using DG.Tweening;

```

```

public static class VFXTweenManager
{
    // Efecto de aparición de partículas con escalado
    public static void PlayParticleBurst(ParticleSystem ps, float duration = 1f)
    {
        ps.Play(); // Inicia el sistema de partículas

        // Escala el transform para dar un efecto de expansión
        ps.transform.localScale = Vector3.zero;
        ps.transform.DOScale(Vector3.one, duration / 2).SetEase(Ease.OutBack);

        // Detiene las partículas tras un tiempo
        DOVirtual.DelayedCall(duration, () =>
        {
            ps.Stop();
        });
    }

    // Encendido de luz con fade de intensidad
    public static void FadeLightIn(Light light, float targetIntensity = 3f, float duration = 1f)
    {
        float originalIntensity = light.intensity; // Guardamos el valor original
        light.intensity = 0f; // Partimos de cero

        // Transición de intensidad con DoTween
        DOTween.To(() => light.intensity, x => light.intensity = x, targetIntensity,
        duration).SetEase(Ease.OutSine);
    }

    // Apagado de luz con fade hacia cero
    public static void FadeLightOut(Light light, float duration = 1f)
    {
        DOTween.To(() => light.intensity, x => light.intensity = x, 0f, duration).SetEase(Ease.InSine);
    }

    // Parpadeo de una luz como efecto de advertencia o carga
    public static void BlinkLight(Light light, float maxIntensity = 5f, int blinks = 3, float blinkDuration =
    0.2f)
    {
        Sequence seq = DOTween.Sequence();

        for (int i = 0; i < blinks; i++)
        {
            seq.Append(DOTween.To(() => light.intensity, x => light.intensity = x, maxIntensity,
            blinkDuration / 2));
            seq.Append(DOTween.To(() => light.intensity, x => light.intensity = x, 0f, blinkDuration / 2));
        }
    }

    // Escalado de partículas con efecto rebote al activarlas
    public static void PunchParticleEffect(Transform particleTransform, float punchScale = 0.5f, float
    duration = 0.6f)
    {

```

```

    particleTransform.DOPunchScale(Vector3.one * punchScale, duration, 10, 1);
}
}

```

Este script `VFXTweenManager` static, con efectos de partículas y luces usando DoTween, tiene los siguientes efectos:

- ★ `PlayParticleBurst`: Emite partículas y las escala con rebote.
- ★ `FadeLightIn / FadeLightOut`: Fundido suave de luces.
- ★ `BlinkLight`: Parpadeo controlado, útil para alertas, poderes, etc.
- ★ `PunchParticleEffect`: Rebote visual al activar un efecto de partículas.

Podemos referirnos a él desde otro lugar tal que así:

```

VFXTweenManager.FadeLightIn(focoLuz);
VFXTweenManager.PlayParticleBurst(particleSystem);

```

=====

Método recomendado para sacudir una cámara en Cinemachine de forma correcta, sin mover el transform del target(sacudimos la mandanga de la cámara) interviniendo en la cámara: `CinemachineBasicMultiChannelPerlin` (el módulo de ruido procedural)

¿Por qué `CinemachineBasicMultiChannelPerlin` ?

Porque no puedes sacudir la cámara directamente (ni su Transform) ya que Cinemachine controla su posición cada frame. Si quieres que “tiemble la imagen” pero sin alterar el enfoque o la posición del objetivo, necesitas introducir ruido desde dentro de Cinemachine. Y eso se hace con este componente..

Configuración:

1. Selecciona tu `CinemachineVirtualCamera` en la escena.
2. Add Extension → `CinemachineBasicMultiChannelPerlin`
3. Configuramos los valores iniciales (pueden estar en cero por defecto):
 - `Amplitude Gain`: controla la intensidad del shake.
 - `Frequency Gain`: controla la rapidez del movimiento (2 o 5).

Código:

```

using Cinemachine;
using DG.Tweening;

```

```

public static class CameraShakeUtility
{
    public static void ShakeCamera(CinemachineVirtualCamera cam, float intensity = 1.5f, float
duration = 0.3f)
    {
        // Obtenemos el componente de ruido
        var perlin = cam.GetCinemachineComponent<CinemachineBasicMultiChannelPerlin>();

        // Verificamos que el componente exista
        if (perlin == null)
        {
            UnityEngine.Debug.LogWarning("La cámara no tiene CinemachineBasicMultiChannelPerlin
agregado.");
            return;
        }

        // Aplicamos la intensidad del shake
        perlin.m_AmplitudeGain = intensity;

        // Esperamos cierto tiempo y luego lo apagamos
        DOVirtual.DelayedCall(duration, () =>
        {
            perlin.m_AmplitudeGain = 0f; // Se detiene el shake
        });
    }
}

```

Ejemplo de uso:

```

public CinemachineVirtualCamera cam;

void OnPlayerHit()
{
    CameraShakeUtility.ShakeCamera(cam, 2.5f, 0.4f);
}

```